

Exercícios — Aula 16 — Introdução a grafos: representação, DFS e BFS

Técnicas de Programação - 2026/02

Última atualização: July 31, 2026

Instruções

- Em lista de adjacência, `adj[v]` contém os vizinhos do vértice `v`.
- Em grafo não direcionado, toda aresta aparece nas duas listas correspondentes.
- Em DFS e BFS, marque o vértice quando ele for descoberto.

Exercício 1 — Simulação de DFS em lista de adjacência

Considere o grafo não direcionado abaixo. A ordem dos vizinhos é a ordem em que aparecem em cada lista.

0: [1, 2]
1: [0, 3]
2: [0, 3]
3: [1, 2, 4]
4: [3]
5: []

Algorithm 1: Dfs

Result: Marca todos os vértices alcançáveis por DFS.
Input : lista de adjacência `adj`, vértice atual, vetor `visitado`
Output: none

```
visitado[atual] ← true
foreach vizinho ∈ adj[atual] do
    if visitado[vizinho] = false then
        | DFS(adj, vizinho, visitado)
    end
end
end
```

Simule `DFS(adj, 0, visitado)`.

	passo	chamada ativa	vértice marcado	próxima chamada	visitados
	1	DFS(0)			
	2				
	3				
	4				
	5				

O vértice 5 é visitado? Por quê?

Exercício 2 — Lista de adjacência

Os vértices são 0, 1, 2, 3 e 4. As conexões são:

0 - 1
0 - 3
1 - 3
2 - 3
3 - 4
4 - 0

1. Monte a lista de adjacência do grafo não direcionado.
2. Monte a lista caso todas as conexões apontem da esquerda para a direita.
3. Em qual versão a conexão entre 0 e 1 aparece em duas listas?
4. Qual é a diferença entre uma aresta direcionada e uma não direcionada?

Exercício 3 — Existe caminho com DFS

Complete o procedimento. Ele deve retornar `true` se o destino puder ser alcançado a partir de `atual`.

Algorithm 2: ExisteCaminho

Result: Verifica alcance entre dois vértices.

Input : `adj`, `atual`, `destino`, `visitado`

Output: boolean

```
if atual = destino then
    | return _____
end
visitado[atual] ← _____
foreach vizinho ∈ adj[atual] do
    | if _____ then
    |     | if ExisteCaminho(adj, vizinho, destino, visitado) then
    |     |     | return _____
    |     | end
    | end
end
end
return _____
```

Teste manualmente sua resposta com origem 0 e destinos 4 e 5 no grafo do Exercício 1.

Exercício 4 — Contar componentes

Complete o procedimento que conta componentes conectadas.

Algorithm 3: ContarComponentes

Result: Conta grupos desconectados de vértices.

Input : lista de adjacência *adj*

Output: int

```
visitado ← ArrayDeFalsos(Tamanho(adj))
componentes ← 0
for v ← 0 to Tamanho(adj) − 1 do
    if _____ then
        DFS(adj, v, visitado)
        componentes ← _____
    end
end
return componentes
```

Use o grafo do Exercício 1. Em quais valores de *v* o contador aumenta?

Exercício 5 — Simulação de BFS

Use o grafo do Exercício 1 e execute BFS a partir do vértice 0. Preencha a tabela.

passo	vértice removido	vértices enfileirados	fila ao final	novas distâncias
1				
2				
3				
4				

Depois indique a distância mínima de 0 até 4 e a distância de 0 até 5.

Exercício 6 — Representação explícita ou implícita

Classifique cada situação e explique como obter seus vizinhos.

Situação	Vértice	Representação explícita ou implícita?	Como obter vizinhos?
labirinto com paredes			
mapa de salas com lista de portas			
rede de pessoas com lista de amizades			
tabuleiro em que peças movem para casas ortogonais			
tabuleiro em que peças movem para casas ortogonais e diagonais			

Explique por que DFS e BFS podem ser usados tanto em uma matriz quanto em uma lista de adjacência.